

L1: 物理内存管理

主要思路

对于多处理器的分配，如果使用一把大锁保护整个管理结构，将导致很长的自旋等待。受实际系统里的buddy

分配器和slab分配器的启发，我把内存分配的过程分为了两个path:

- slow path

某个cpu申请物理页大小级别的内存。此时用一把大锁锁住buddy的管理结构，从中分配出页面。

- fast path

某个cpu申请小于物理页大小级别的内存。此时先查询该cpu自己的slab缓存里还有没有可用的piece，如果有就将一个piece从链表里摘下来分配出去；如果没有，再走slow path，从buddy分配器里获取一个物理页。将此页划分出一些空间用于储存管理结构之后，再将其切成若干个对应大小的piece，分配一个piece出去，剩余的piece留在该cpu的slab缓存链表里，留给后续使用。

归还方式也有所区分:

由于slab开头需要一些头信息，故slab分配器分配的内存总是不和物理页对齐，很容易区分出来内存是哪个分配器分出去的。

- buddy分配器分配出去的内存

一把大锁锁住buddy的管理结构，正常归还到buddy分配器。

- slab分配器分配出去的内存

直接把所归还的piece放回当前cpu的slab缓存链表里。由于一个cpu申请的内存可能在其他cpu上被释放，这里的归还方式有可能使得某个cpu“偷取”了另一个cpu的piece。不过这并不要紧，由于slab分配上有一些特别的地址结构设计，一个piece的管理结构和他真正管理的空间可以轻易地互相定位，而并不需要知道是哪个cpu持有着它。

如果一个slab所切成的全部piece都被归还，我选择把他继续留在cpu的slab缓存里而不是归还buddy。因为归还buddy就违背了fast path的初衷——workload可以频繁申请一个小内存再释放之，使得cpu忙碌于反复地从buddy申请/归还。

一些精巧的设计

区分buddy或slab

分配时分配器是我们定的，但归还时cpu仅仅交给我们一个地址，我们需要区别它应如何归还。

由于buddy机制按物理页为单位分配内存，分配出去的地址总是和页面对齐。而slab机制是先从buddy里取一个页，用去头部一些空间后再划分piece，其分配出去的地址总是不和页面对齐。我们可以以此来区别二者：

```
static void kfree(void *ptr) {
    if (((uintptr_t)ptr) & (PAGE_SIZE - 1)) == 0) {
        // aligned to page, it must be allocate by buddy
        buddy_free(ptr);
    } else {
        slab_free(ptr);
    }
}
```

充分榨干堆空间

由于堆的头部空间被拿来存管理结构，同时又要照顾到对齐的要求，堆空间的形状可能会变成下面这样：

```
| management structure | padding | available space | free space |
```

其中padding是为了对齐必须保留的，而free space还可以继续利用(由于buddy机制的需要，最开始available space是2的幂，没有往后拓展)。于是我从最大的2的幂开始不断向下探索，如果padding还够management structure吃掉，且free space还够available space吃掉，就往后拓展。新吃掉的空间放入buddy对应level链表里。

管理结构与真正空间的互定位

不管是slab还是buddy，我都会在其管理空间的头部存下一些信息，这些信息可以帮助管理结构定位到其真正管理的空间。而对于另一个方向的定位，我则通过地址结构上的设计，使一个地址能轻松找到其管理结构的头部，从而定位到其管理结构。

印象深刻的Bug

我搭建测试框架的时候写了一个C++程序checker，读入若干次kalloc、kfree的记录，判断其中是否有矛盾：

- kfree一个还没kalloc的内存区间
- kalloc的内存区间与未被kfree的另一个内存区间相交

结果我如愿得到了assertion failed。再反复排查之后我坚信自己的分配代码没有问题。最后我发现了我输出kalloc、kfree日志的方式有问题——我的输出方式类似于：

```
void *ptr = pmm->alloc(size);
printf("kalloc %p %p\n", ptr, ptr + size);
```

由于我把上锁的过程封装到alloc里面了，所以很长时间我都没有意识到，printf并未和alloc锁在一起，这也就意味着可能申请的空间都被释放了，我还没能输出我的kalloc日志，所以就导致了checker的assertion failed。在fix了这个bug之后，我成功通过了checker测试。

这个Bug算是让我体会到了并发Bug的无处不在，以及它超强的隐蔽性——在上锁的代码被封装到函数里面的时候更是如此。

L2: 内核线程管理

代码架构

本次实验采用的代码架构与实验手册所给的架构基本一致。使用 `on_irq` 函数来注册中断处理函数，使 `os_trap` 保持简单。

```
static void kmt_init() {
    os->on_irq(INT32_MIN, EVENT_NULL, kmt_context_save);
    os->on_irq(INT32_MAX, EVENT_NULL, kmt_schedule);
    ...
}
```

同时添加了本地测试部分：

```

sem_t empty, fill;
#define P kmt->sem_wait
#define V kmt->sem_signal

void T_produce(void *arg) {
    while (1) { P(&empty); putchar('('); V(&fill); }
}

void T_consume(void *arg) {
    while (1) { P(&fill); putchar(')'); V(&empty); }
}

static void run_test1() {
    int N = 5;
    int NPROD = 1;
    int NCONS = 1;
    kmt->sem_init(&empty, "empty", N);
    kmt->sem_init(&fill, "fill", 0);
    for (int i = 0; i < NPROD; ++i) {
        kmt->create(task_alloc(), "producer", T_produce, NULL);
    }
    for (int i = 0; i < NCONS; ++i) {
        kmt->create(task_alloc(), "consumer", T_consume, NULL);
    }
}

static void os_init() {
    os_init_handlers();
    pmm->init();
    kmt->init();
    ...
#ifdef LOCAL_TEST
    run_test1();
#endif
}

```

印象深刻的Bug

自旋锁的修改

在课上展示的代码中：

```
void push_off() {
    int old = ienabled();
    struct cpu *c = mycpu;
    iset(false);
    ...
}
```

c 被赋值的过程有可能被中断，正确的写法应该是这样：

```
void push_off() {
    int old = ienabled();
    iset(false);
    struct cpu *c = mycpu;
    ...
}
```

这样就能保证 `iset(false)` 以下的部分都不会被中断执行。

os_trap 栈空间

Abstract Machine的规约里写道：

在异常 (trap) 发生后，会借用当前执行流的栈，在栈上保存额外的数据后关闭中断，然后**直接**
在当前栈上调用 `cte_init` 时注册好的 handler 函数。handler 返回一个指向执行流的指针，并切
换到该执行流执行。

这就意味着，即使中断处理函数已经选出了要切换到的下一个 `task`，`os_trap` 函数仍然还在使用当前 `task` 的栈空间，且从 `os_trap` 返回到上下文真正切换完毕也有一段AM的代码在使用当前 `task` 的栈空间。如果在这段时间，当前 `task` 因为被标记为已换下，被其他cpu选中执行，两个cpu就会在同一个栈空间执行代码，造成并发bug。

解决这个问题使用了“延后处理”的思路。即当我们要从cpu上换下一个 `task` 时，并不马上标记 `task` 可用，而是直到下一个 `task` 真正跑起来以后，也即我们已经用上新的栈空间之后，再标记上一个 `task` 可用。可以发现，我们需要hook“新的 `task` 跑起来”这个事件。

由于我们没法hack AM，只能尝试别的思路。例如，新的 `task` 运行一段时间后一定会被中断，我们可以在此时把上一个 `task` 标记为可用。

相关代码：

```
static Context *kmt_context_save(Event ev, Context *context) {
    int c = cpu_current();
    kmt->spin_lock(&lock_tasks_list);

    current[c]->context = *context;
    assert(task_buf[c] == NULL);
    // 缓存下来, 并不马上标记为可用
    task_buf[c] = current[c];
    // current[c]->status = TASK_RUNABLE;

    kmt->spin_unlock(&lock_tasks_list);
    return NULL;
}
```

```
static Context* os_trap(Event ev, Context *context) {
    int cur = cpu_current();
    if (task_buf[cur]) {
        // 此时标记上一个task为可用
        task_buf[cur]->status = TASK_RUNABLE;
        task_buf[cur] = NULL;
    }
    ...
}
```